



Fundusze
Europejskie
Wiedza Edukacja Rozwój



Rzeczpospolita
Polska

Politechnika
Warszawska

Unia Europejska
Europejski Fundusz Społeczny



Algorytmy w Inżynierii Danych

julia * FTW

Bartosz Chaber

[https://www.tesla.com/sites/default/files/
images/careers/autopilot/network.mp4](https://www.tesla.com/sites/default/files/images/careers/autopilot/network.mp4)

Splot / Konwolucja

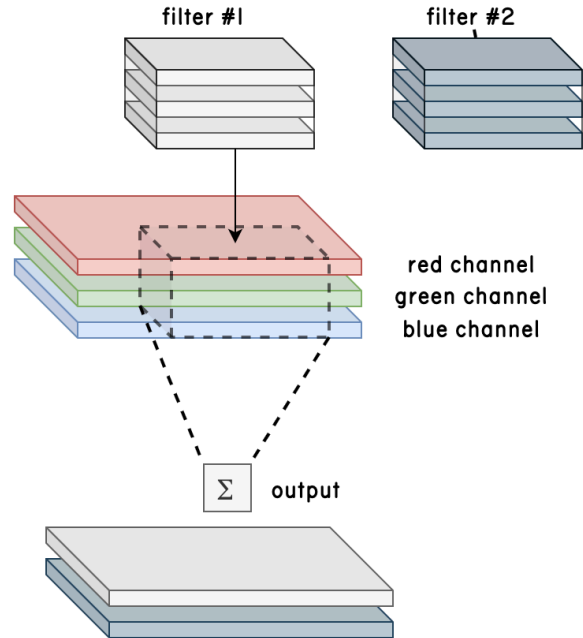
Czasem nazywane też mnożeniem splotowym:

$$f(t) * g(t) = \int_{-\infty}^{\infty} f(t - \tau)g(\tau) d\tau$$

Zwykle, rozpatrujemy to na spróbkowanych, dyskretnych wartościach funkcji:

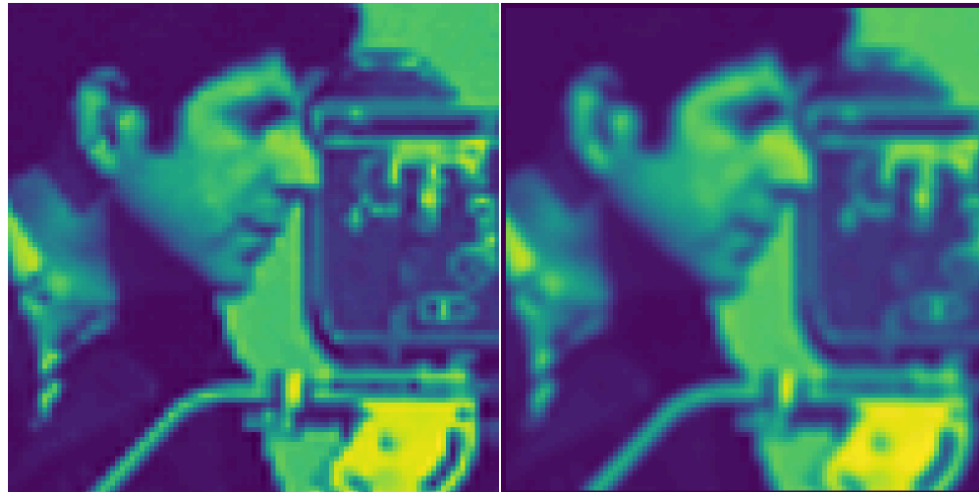
$$f_n * g_n = \sum_{m=-\infty}^{\infty} f(n - m)g(m)$$

Na dzisiejszych zajęciach przyjrzymy się algorytmom obliczającym konwolucję obrazów.



Filtry obrazów jako wynik konwolucji

Jednym z najbardziej naturalnych zastosowań (również wykorzystywanych w uczeniu maszynowym) jest konwolucja obrazów z pewnymi *maskami/filtrami*. Po angielsku nazywa się je czasem *kernel*.



Obsługa brzegu (ang. padding)

Wielokrotnie spotykamy się z sytuacją, gdy filtr “wystaje” poza nasz obraz. Mamy kilka sposobów uwzględniania tego, co jest poza naszym obrazem.

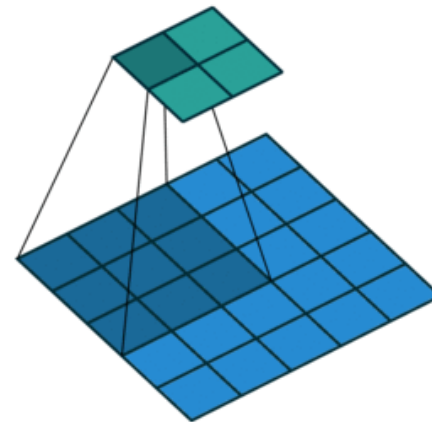
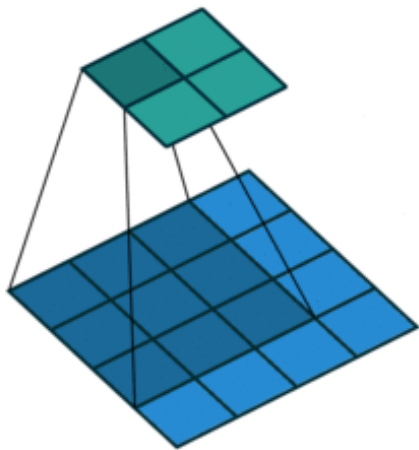
zero-padding									
0	0	0	0	0	0	0	0	0	0
0	2	3	4	5	6	0	0	0	0
0	7				2	0	0	0	0
0	8				4	0	0	0	0
0	9				6	0	0	0	0
0	8	6	4	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

edge									
2	2	3	4	5	6	6	6	6	6
2	2	3	4	5	6	6	6	6	6
7	7				2	2	2	2	2
8	8				4	4	4	4	4
9	9				6	6	6	6	6
8	8	6	4	2	0	0	0	0	0
8	8	6	4	2	0	0	0	0	0

wrap									
0	8	6	4	2	0	8	8	8	8
6	2	3	4	5	6	2	2	2	2
2	7				2	7	7	7	7
4	8				4	8	8	8	8
6	9				6	9	9	9	9
0	8	6	4	2	0	8	8	8	8
6	2	3	4	5	6	2	2	2	2

Tryby operacji splotu

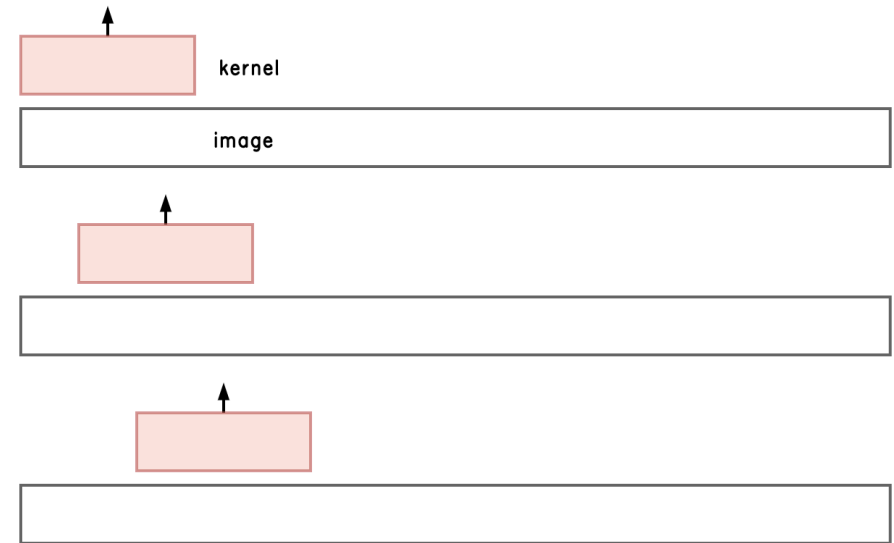
Vincent Dumoulin, Francesco Visin - A guide to convolution arithmetic for deep learning https://github.com/vdumoulin/conv_arithmetic



Naiwna implementacja

Najprostsza implementacja wymaga pętli po kolejnych ustawieniach maski splotu nad obrazem.

```
function conv(I::Vector,  
             K::Vector)  
    n = length(I) - 1  
    J = zeros(n+1)  
    for i=2:n  
        J[i] = sum(I[i-1:i+1] .* K)  
    end  
    return J  
end
```

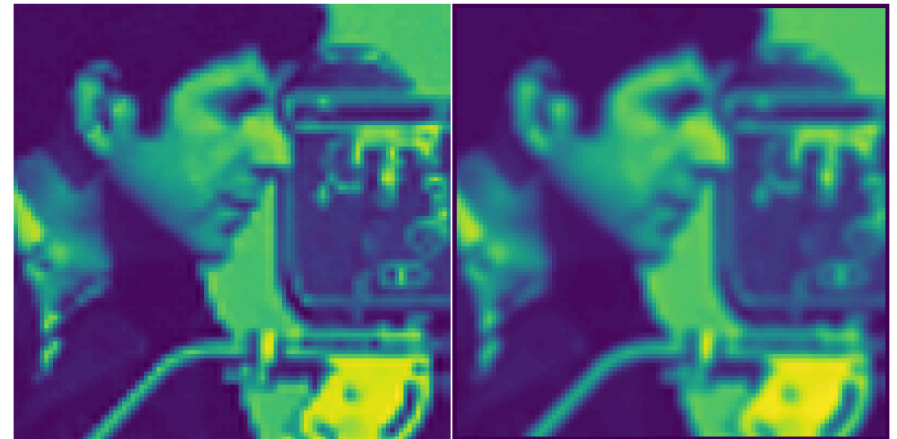


Rozmywanie i wykrywanie krawędzi

Różne maski mają różne działanie:

```
function conv(I::Matrix,
              K::Matrix)
    n, m = size(I) .- 1
    J = zeros(n+1, m+1)
    for i=2:n, j=2:m
        J[i, j] =
            sum(I[i-1:i+1, j-1:j+1] .* K)
    end
    return J
end
```

```
conv(img, [1/9 1/9 1/9
           1/9 1/9 1/9
           1/9 1/9 1/9])
```



Różne maski mają różne działanie:

```
function conv(I::Matrix,  
              K::Matrix)  
    n, m = size(I) .- 1  
    J = zeros(n+1, m+1)  
    for i=2:n, j=2:m  
        J[i, j] =  
            sum(I[i-1:i+1, j-1:j+1] .* K)  
    end  
    return J  
end
```

Wykrywanie krawędzi (filtr
Sobela):

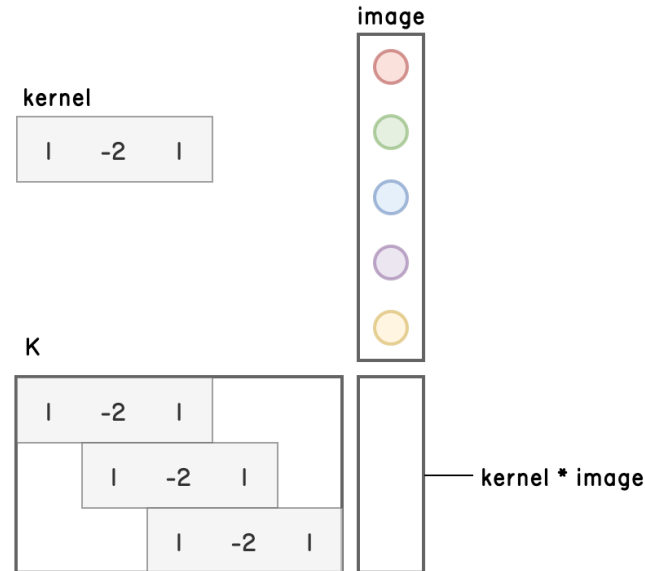
```
conv(img, [-1 -2 -1  
           0  .0  0  
           +1 +2 +1])
```



Splot jako mnożenie macierzowe

W ogólności możemy zamienić pętle po położeniach maski na mnożenie macierzy przez obraz:

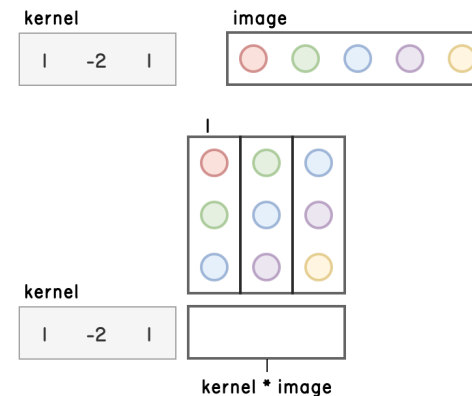
```
function mulconv(img, ker, n, m)
    I = vec(img)
    K = spdiags(
        0   => ker[2,2]*ones(n*m),
        -1  => ker[2,1]*ones(n*m-1),
        +1  => ker[2,3]*ones(n*m-1),
        -m   => ker[1,2]*ones(n*m-m),
        +m   => ker[3,2]*ones(n*m-m),
        -m-1 => ker[1,1]*ones(n*m-m-1),
        -m+1 => ker[1,3]*ones(n*m-m-1),
        +m-1 => ker[3,1]*ones(n*m-m-1),
        +m+1 => ker[3,3]*ones(n*m-m-1))
    reshape(K * I, n, m)
end
```



Splot jako mnożenie macierzowe (im2col)

Możemy też zmodyfikować macierz związaną z obrazem:

```
# Author: Seif_Shebl
# URL: https://discourse.julialang.org/t/
#      what-is-julias-im2col/14066/9
@inline function im2col(A, n, m)
    M,N = size(A)
    B = Array{eltype(A)}{undef, m*n,
        (M-m+1)*(N-n+1)}
    indx = reshape(1:M*N, M,N)[1:M-m+1,1:N-n+1]
    for (i,value) in enumerate(indx)
        for j = 0:n-1
            @views B[(i-1)*m*n+j*m+1:(i-1)*m*n+(j+1)*m] =
                A[value+j*M:value+m-1+j*M]
        end
    end
    return B'
end
```



https://leonardoaraujosantos.gitbook.io/artificial-intelligence/machine_learning/deep_learning/convolution_layer/making_faster

Splot w dziedzinie częstotliwości

Ostatnia, prezentowana dzisiaj metoda polega na wykorzystaniu twierdzenia o konwolucji, tj. splot w dziedzinie czasu jest równoważny mnożeniu (*element-wise*) w dziedzinie **częstotliwości**.

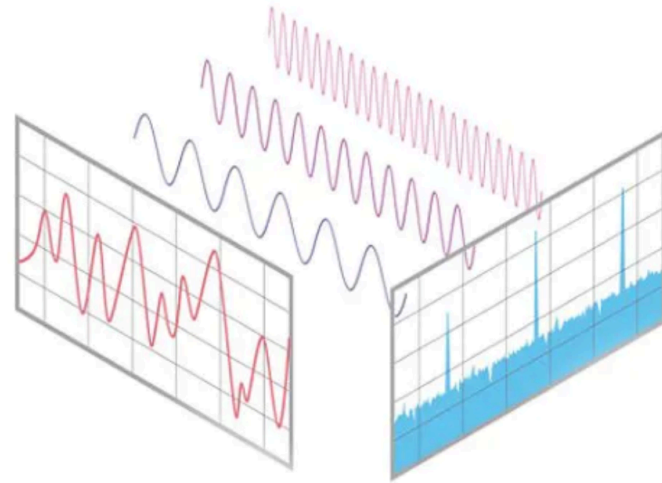
F : operator transformujący między dziedziną czasu, a dziedziną częstotliwości. F^{-1} : operator transformujący między dziedziną częstotliwości, a czasu.

$$F = F(f),$$

$$G = F(g).$$

$$f * g = F^{-1}(F \cdot G)$$

Oznacza to, że jeżeli potrafimy *szybko* dokonać transformacji między dziedziną czasu a częstotliwości to *opłaca* się nam dokonać splotu w dziedzinie częstotliwości.



źródło: fragment logo EasyFFT Frequency Transform for Arduino

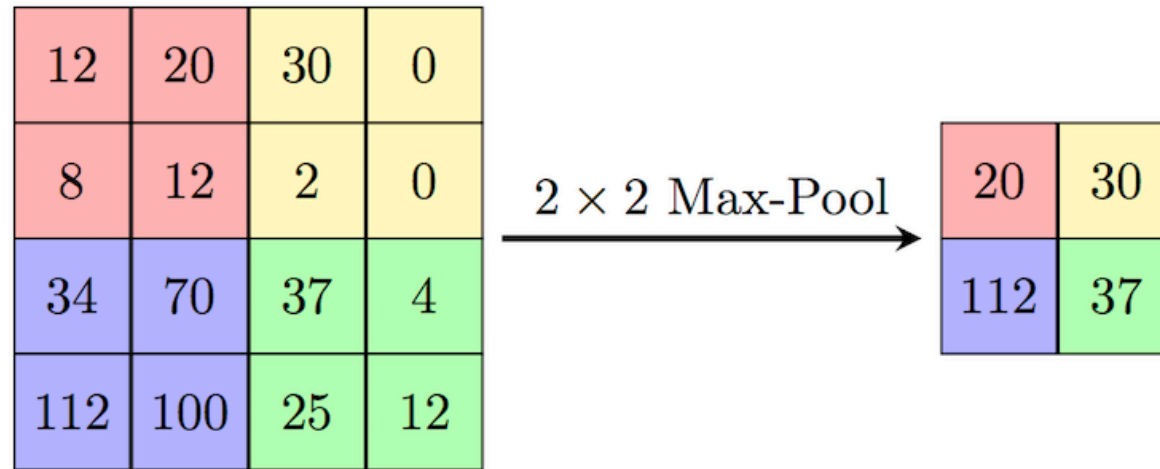
Umiemy szybko wykonywać transformację Fouriera!

FFT (Fast Fourier Transform) polega na liczeniu dyskretnej transformaty Fouriera dla fragmentów sygnału. Powszechnie wykorzystywany jest wariant, w którym dzielimy fragmenty sygnału na dwa (dwukrotnie krótsze sygnały) aż operujemy tylko na dwóch próbkach. Dlatego największą efektywność uzyskujemy dla sygnałów o liczbie próbek $N = 2^k$.

```
function  
fftconv(img::Matrix,  
        kernel::Matrix)  
    ker = zero(img)  
    ker[1:3, 1:3] .= kernel  
    I, K = fft(img), fft(ker)  
    J = ifft(I .* K)  
    return abs.(J)  
end
```

Operacja MaxPool

Zwykle maski warstw splotowych nie są zbyt duże, więc nie potrafią uwzględnić wpływu dalszych elementów obrazu. Dlatego w sieciach CNN często spotyka się warstwy “obniżające” rozdzielczość (efektywnie poszerzające maski dla splotu). Jedną z takich warstw jest MaxPool. Wynik tej warstwy zmniejsza zwykle liczbę wierszy/kolumn co najmniej dwukrotnie.



Jak zróżniczkować warstwę MaxPool?

(na tablicy)

Literatura

1. Leonardo Araujo dos Santos, Convolution Layer – Making faster, URL:

https://leonardoaraujosantos.gitbook.io/artificial-intelligence/machine_learning/deep_learning/convolution_layer/making_faster, dostęp: 07.04.2024

2. Vincent Dumoulin, Francesco Visin, A guide to convolution arithmetic for deep learning, 2018, arXiv 1603.07285, stat.ML.